

Tenets of Embedded Systems Security

Goals and Objectives

Defining a Tenet

Merriam-Webster definition

noun te·net \ ˈte-nət *also* ˈtē-nət \

A principle, belief, or doctrine generally held to be true; *especially*: one held in common by members of an organization, movement, or profession.

Secure Boot—Firmware Integrity

- One of the most critical elements to ensure the integrity of processes running on a target device
- Ensures that the code is validated and authenticated prior to execution
- Generally accomplished through one or more of the following mechanisms:
  - Trusted computing modules
  - Digitally signed binaries
  - Boot level encryption
  - Trusted bootloaders



Secure Storage of Critical Data

- Ensure secure storage of critical data to protect device operation including:
  - Firmware or other binaries detailed previously
  - Data stored by the device itself
- May include:
  - User data
  - Intellectual property
  - Cryptographic material
  - Set points and configuration data



In-Service Firmware Upgrade Integrity

- Enhance firmware upgrade process to support and facilitate:
  - Modification of the system binaries interacting with secure boot mechanisms
- Firmware upgrades should:
  - Be obtained from a verifiable source
  - Be validated before application
  - Not expose critical data during upgrade process
  - Fail securely to a known-good state

Secure Communications

- Securely transmit data that:
  - Requires protection of its confidentiality, authenticity, and integrity to be consistent with:
    - Security objectives
    - Device capabilities
  - Includes data transmitted between components and onboard (inter and intra-component)
    - For example, code decrypted in memory and transmitted in the clear for processing



Implementation Flaws Minimized

Generic software flaws or implementation errors can undermine the efficacy of additional controls. Incorporate in the development lifecycle:

- Code scanning
- Peer review
- Other traditional software security efforts

To ensure interaction between components has not been compromised by implementation flaws, also include:

- Third-party testing
- System-level tests

Identity and Access Management

- Know which device is which
  - Shared keys do not achieve this
- Know who can connect to which devices
- Limit access to authorized users
  - Applies to functionality: physical and logical
- Manage devices and access on the back end

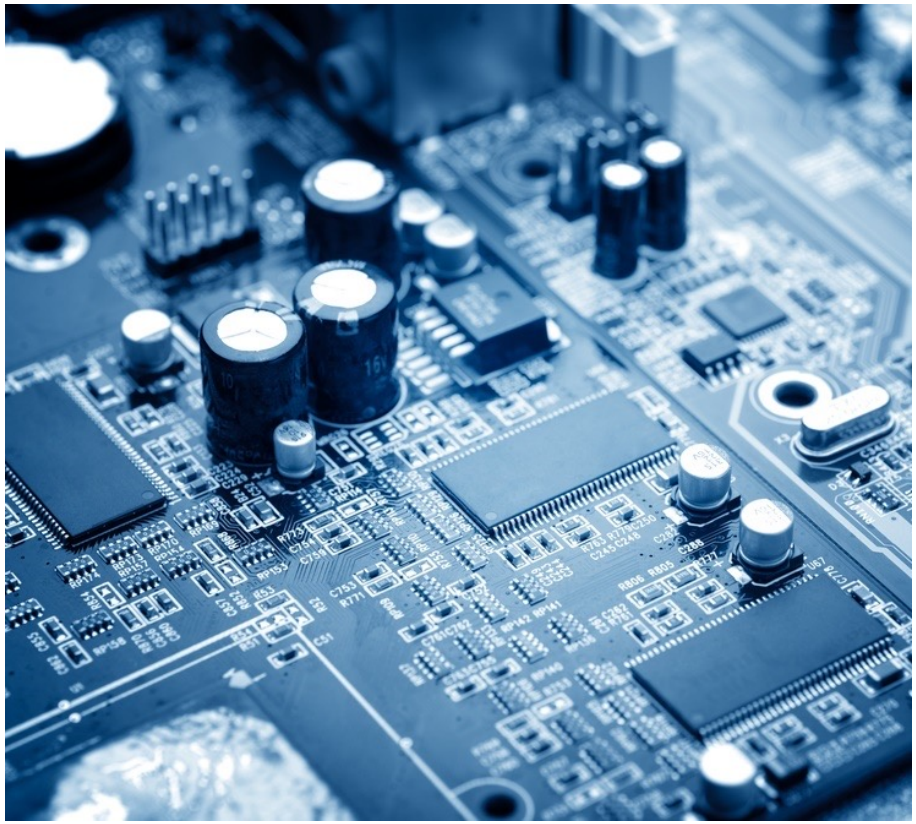


Interface Control

Control of external interfaces includes:

- Management ports
- Debug ports
- Maintenance interfaces and functionality
- Test interfaces and functionality

This is critical towards ensuring that external access to the device is managed securely.



Managing External Elements

- External elements include how other products, persons, and related systems interact with the the in-scope system element
- Ensure open source or vendor-supplied components (software, hardware, or system) modifications or vulnerabilities are managed within the lifecycle of the device

Avoid “Security through Obscurity”

- Relying on an attacker failing to learn of a trivial security method is a great way to fail

- Kerckhoff's Principle and Shannon’s Maxim state:
  - “A cryptosystem should be secure even if everything about the system, except the key, is public knowledge”

  - “One ought to design systems under the assumption that the enemy will immediately gain full familiarity with them”



Wrapping the Concepts Together

Managing Risk across the System

- Risk is always non-zero
- Security design, especially in embedded systems, should carry the objective to make attacks, ”Academic”

Addressing Common Vulnerability Issues

As far back as in 2011, CWE published a set of “monster mitigations.”

Monster Mitigation Index

ID	Name
M1	Establish and maintain control over all of your inputs.
M2	Establish and maintain control over all of your outputs.
M3	Lock down your environment.
M4	Assume that external components can be subverted, and your code can be read by anyone.
M5	Use industry-accepted security features instead of inventing your own.
GP1	*(general)* Use libraries and frameworks that make it easier to avoid introducing weaknesses.
GP2	*(general)* Integrate security into the entire software development lifecycle.
GP3	*(general)* Use a broad mix of methods to comprehensively find and prevent weaknesses.
GP4	*(general)* Allow locked-down clients to interact with your software.

Addressing Common Vulnerability Issues (Continued)

IEEE guidelines on how to avoid the top 10 design issues:

1. Earn or give, but never assume, trust.
2. Use an authentication method that cannot be bypassed or tampered with.
3. Authorize after you authenticate.
4. Strictly separate data and control instructions, and never process control instructions received from an untrusted source.
5. Use cryptography correctly.
6. Identify sensitive data and how they should be handled.
7. Always consider the users.
8. Understand how integrating external components changes your attack surface.
9. Be flexible when considering future changes to objects and actors.
10. Get Involved.

Analysis and Discussion

Security Tenets of Embedded Systems Summary

Secure Boot

External Elements

Security Through Obscurity

Identity Management

Interface Control

Secure Communications

Implementation Flaws

Firmware Upgrade

Secure Storage

Thank You!

